IIT Bombay
Aerospace Department
Supervised Learning Project

Novel Method to Classification Problems And their Spatial Interpretability

Guide -
Prof. Prabhu Ramachandran
Co-guide -
Prof. Harshad Khadilkar

Author-
Samarth Pusalkar
19D180029

Index

Classification Problems-

Classification Problems are common in areas of machine learning and here in this documentation they refer to marking a set of inputs or data points to a unique label or class.

Classification Standard Procedures-

A briefing and their problem to address

Support Vector Machine -

This mainly involves a projection of input data points to higher dimensions and then finding a hyperplane that separates the classes spatially.

Bayes Classifier

Involves maximizing the probability that input belongs to a particular class-

$Pr(Y = j|X = x_0)$

And the logistic classifier / multi class logistic regression models which tend to approximate the bayes decision boundary that separates two classes.

$$\Pr(Y = k|X = x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}$$

KNN Classifier -

Using the k nearest points to judge what class an input belongs →
This project is an extension of the KNN classifier first to an unsupervised setting and judging if the modification to the original KNN does any better then turning the model to supervised setting so it can be used out of the box to predict labels for new inputs.

Major problem to address in all these methods is training/predicting computation expense and time which we will try to reduce.

Some points to note-

In areas of machine learning that revolve around prediction - they involve regressing the true function relating the input to outputs to newer data points. Before we proceed it must be stated that training any model that can do such prediction, requires data points or training data, while

we can interpolate the training data points to the test data points that are within the domain covered by the training data it is generally not possible to accurately extrapolate the training data points beyond the training domain unless in special cases when the nature of the model used to fit the data captures the true nature of the real relation between input and output, eg. a linear relation between input-output can be easily captured and extrapolated by a linear model but not extrapolated by any general model like cubic spline or logistic regression.. As the nature of the input-output in general is not known, we therefore are going to use general fitting methods that work on all data, and keeping this in mind our major focus will be on interpolation of the data and not on approximating the true nature of the function or prediction beyond training domain.

Interpolation

There are two types of data interpolation that machine learning concerns -

1) Regressive or continuous values prediction values
2) Categorical values prediction

First we look for a simple case of interpolation of regressive data using cubic splines:

Numeric Interpolation::

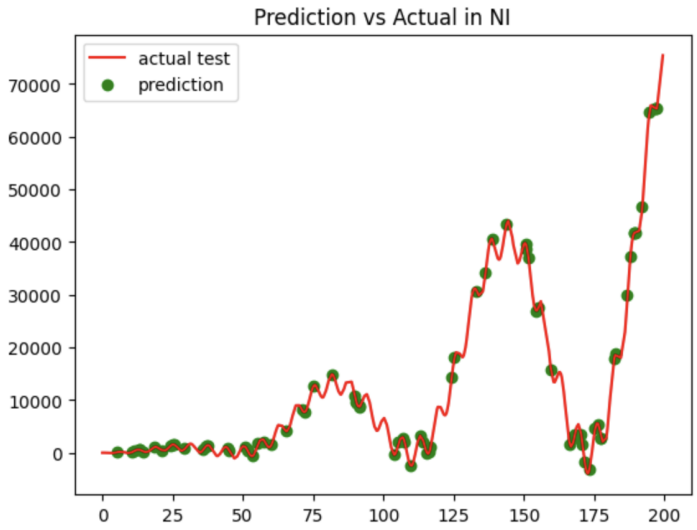
We take the nearest 4 points of interest to the input/test data point and then use them to fit a cubic curve that best works as a natural interpolation of the training data.. We take the average slope and value information of the two points on either side of the test point, then with this 4 sets of equations fit the cubic equation..
Here what we mean by natural interpolation is that the slope of the approximating function remains continuous when we interpolate the data points, while it may not be differentiable.

Testing on random functions and data points:

```
f = lambda x: np.e**np.sin((x**2))+np.sin(x/10)*abs(x**2-4)+((x**2)+23*x*(np.cos(x)))+abs(np.cos(x))**(x**2)

x_train = np.linspace(0,200,800)
X_train = x_train.reshape(-1,1)
Y_train = f(X_train)
x_test = np.array(sorted(200*np.random.rand(500,1)))
X_test = x_test.reshape(-1,1)
Y_test = f(X_test).reshape(-1,)
def interpolt(X,Y,inpus):
    inpus = np.array(inpus).reshape(-1,)
    Ans = []
    for inp in inpus:
        inpu = np.array([inp]).reshape(-1,1)
        Y = Y.reshape(-1,1)
        xx = (X[1:]-X[:-1]).reshape(-1,1)
        yy = Y[1:]-Y[:-1]
        tt = (yy/xx**1.1)
        t_ = 0*X
        rr = X.shape[-1]-2
        t_[1:-1] = ((tt[:-1]+tt[1:])/((np.pi-np.e)*2)/(1+0/xx[:-1]+0/xx[1:])).reshape(-1,1)
        t_[0] = tt[0]
        t_[-1] = tt[-1]
        tree = KDTree(X.reshape(-1,1))
        distances, indices = tree.query(inpu,min(10,len(X)))
        sd = [0]
        _2 = indices.reshape(-1,)[(X[indices[0]]>inpu).reshape(-1,)]
        _2 = _2[0] if len(_2) else -1
        _1 = indices.reshape(-1,)[(X[indices[0]]<inpu).reshape(-1,)]
        _1 = _1[0] if len(_1) else 0

        if _1==_2:
            Ans.append(Y[_2][0])
            continue

        x1 = X[_1]
        x2 = X[_2]
        M = np.array([[x1**3,x1**2,x1,1],[x2**3,x2**2,x2,1],[3*x1**2,2*x1,1,0],[3*x2**2,2*x2,1,0]])
        Vec = np.array([Y[_1],Y[_2],t_[_1],t_[_2]])
        try:
            pp = np.dot(np.linalg.inv(M),Vec)
        except:
            Ans.append(Y[_2][0])
            continue
        Ans.append(np.polyval(pp,inpu)[0][0])
    return np.array(Ans)
```

```
y_pred = interpolt(X_train,Y_train,x_test)
plt.plot(x_test,Y_test,color ="red",label="actual test")
filte = np.random.randint(0,len(y_pred),80)
plt.scatter(x_test[filte],y_pred[filte],color="green",label="prediction")
plt.title("Prediction vs Actual in NI")
plt.legend()
print("RMS of relative error",((((Y_test-np.array(y_pred))/Y_test)**2).mean())**0.5)
```

```
RMS of relative error 1.9866217978795675
```



We note while playing around different functions and training data size that increasing the data points as expected increases the fitting quality (we are not quantifying this at this point)

Categorical Interpolation::

For categorical classification problems we modify the KNN algorithm and instead find the weighted average of the labels or categories of points nearby, here weights are a function of distance of the training data point from point of interest, the function can be anything but for the case of implementation we are using something similar to logistic function as we need to capture the importance of nearer points over far away points.

The Algorithm

The algorithm is similar to KNN, but instead of taking the discrete values of classes and averaging them, they are
averaged by weighted by the distance it is from the point of interest.. Simple weighted average based on distance does not provide the best result so the weight is distance passed through a sigmoid/exponential function. Taking all the points in the training data is irrelevant as the weight value decreases with increasing distance so instead some nearby points are considered. The

number of such points under consideration is currently arbitrary but it depends on the density of points in the training data.

The steps::

Consider the input X, we find k nearest points to X in the training data

Let $d_1, d_2, d_3, d_4..d_k$ be the distances and $y_1, y_2, y_3, y_4..y_k$ be the distances and respective classes of the training data points, respectively.

We take a function f(x) (=e^(-x), or logistic function(works better)).. that converts distances to weights for averaging..

We find
$y_{pred} = (\Sigma f(d_i) * y_i)/(\Sigma f(d_i))$

This is the prediction of the input data.

```
def KNN_Classify(x_train,Y_train,inpus):
    Ans = []
    n = len(x_train)
    filteTrain = [np.random.randint(0,x_train.shape[0],min(7000,int(n/1)))]
    X_train = x_train[filteTrain]
    y_train = Y_train[filteTrain]
    tree = KDTree(X_train)
    distances, indices = tree.query(inpus, k=min(700,int(n/3)))
    t = 0
    for ii in range(len(inpus)):
        inpu = inpus[ii]
        dis = distances[ii]
        bias = y_train.mean(axis=0)
        bb = bias.mean()
        k = 2*np.pi
        bias-=bb
        DD = ((1/(np.e**(k*dis)+1).reshape(-1,1))).sum()
#         print(DD)
#         print(t,"_________________",t,flush=True)
        t+=1
        Ans.append((((y_train[indices[ii]])/((np.e**(k*dis)+1).reshape(-1,1))).sum(axis=0)/DD)-bias)
    return Ans
```

```
## Loading MNIST DATASET

(x_train, Y_train), (x_test, Y_test) = mnist.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0
x_train = x_train.reshape(60000,-1)
Y_train = Y_train.reshape(60000,1)
Y_train = to_categorical(Y_train)
x_test = x_test.reshape(10000,-1)
Y_test = Y_test.reshape(10000,1)
Y_test = to_categorical(Y_test)
```

```
## Predicting the Classes for the Test Set

filteTest = [np.random.randint(0,x_test.shape[0],1000)]
X_test = x_test[filteTest]
y_test = Y_test[filteTest]
y_pred = np.array(KNN_Classify(x_train,Y_train,X_test))
# np.argmax(Y_train,axis=1)

## Checking Accuracy
n = len(y_test)
acc = sum([np.argmax(y_test[i])==np.argmax(y_pred[i]) for i in range(n)])/n
ind = np.arange(0,n)
print("The Accuracy for this test set")
acc
```

```
The Accuracy for this test set
0.932
```

```
### Testing on a different dataset

X, Y = sklearn.datasets.load_iris(return_X_y=True)
x_train,x_test,Y_train,Y_test = train_test_split(X,Y)
Y_train = to_categorical(Y_train)
Y_test = to_categorical(Y_test)
```

```
## Predicting the Classes for the Test Set

filteTest = [np.random.randint(0,x_test.shape[0],3000)]
X_test = x_test[filteTest]
y_test = Y_test[filteTest]
y_pred = np.array(KNN_Classify(x_train,Y_train,X_test))
```

```
## Checking Accuracy
n = len(y_test)
acc = sum([np.argmax(y_test[i])==np.argmax(y_pred[i]) for i in range(n)])/n
#acc = 1 - sum([abs((y_test[i])-(y_pred[i])) for i in range(n)])/n

print("The Accuracy for this test set")
acc
```

```
The Accuracy for this test set
0.947
```

```
### Testing on a different dataset

data = sklearn.datasets.fetch_openml("diabetes", version=1, as_frame=True, return_X_y=False)
X = np.array(data["data"])
Y = np.where(data["target"]=="tested_positive",0,1)#np.array(list(map(list(data["target"]),lambda x: 1 if x == "test
x_train,x_test,Y_train,Y_test = train_test_split(X,Y)
Y_train = to_categorical(Y_train)
Y_test = to_categorical(Y_test)
```

```
## Predicting the Classes for the Test Set

filteTest = [np.random.randint(0,x_test.shape[0],3000)]
X_test = x_test#[filteTest]
y_test = Y_test#[filteTest]
y_pred = np.array(KNN_Classify(x_train,Y_train,X_test))
```

```
## Checking Accuracy
n = len(y_test)
acc = sum([np.argmax(y_test[i])==np.argmax(y_pred[i]) for i in range(n)])/n
#acc = 1 - sum([abs((y_test[i])-(y_pred[i])) for i in range(n)])/n

print("The Accuracy for this test set")
acc
```

```
The Accuracy for this test set
0.71875
```

We find that we are able to achieve substantial accuracy of different classification, above 95% (with parameters tweaked) on MNIST dataset and similarly on “titanic” and “diabetes” dataset.

This high accuracy suggests an important thing to note is spatial closeness of similar classes. Classes that are similar or the same are spatially closer to each other, here distance used is euclidean.

Problem Faced by this Approach:

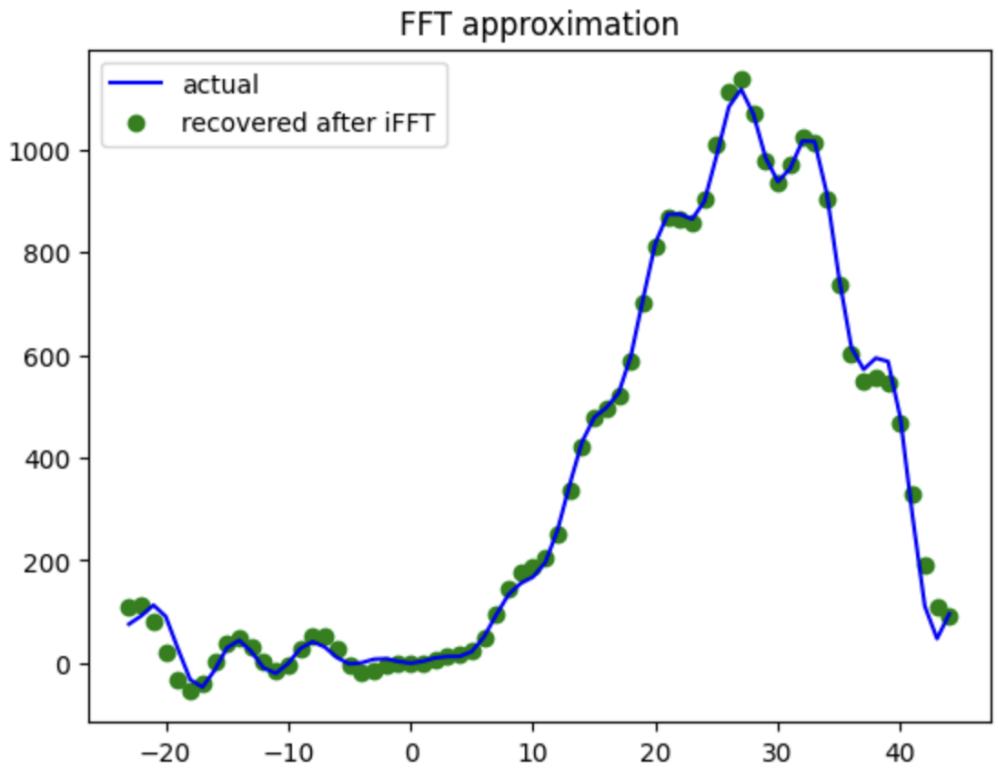
An obvious flaw which increasingly becomes an important issue for larger training data is that even though this does not require any explicitly used for any classification without modification, is that this requires peeking to the training data every time a prediction has to be made, so that involves storing training data and also time to find nearest points in the training data to reduce computation time, which becomes inefficient for larger training data.
One might think to reduce the training data to some few important data points that would capture the essence and avoids the problem of larger training data however this method is not scalable to even larger training data, other extensions could be to make a logistic approximation of weights, but when implemented it fails to capture the spatial information that separates different data points..

Solution::

We need to find a resolution to this problem that does not compromise much on the positive side of not requiring any training time..
Initially we try to compress the training data using fourier transform and then just storing some relevant or most dominating frequencies then using them to compute the inverse transform reproducing the required data on demand..

```
def packageItFFT(Y,hp=10):
    filtr = (1+0*Y).astype(bool)
    ft = np.fft.fftn(Y)
    ff = ft.reshape(-1,)
    filtr = (0*ff)
    filtr = ((abs(ff))>max(abs(ff))/hp)
    sd = (ff*0+0j)
    sd[filtr] = (ff[filtr])
    return np.where(filtr)[0],ff[filtr],ft.shape
def depackageFFT(sshape,filtr,fill,k=-1):
    sd = 0*np.zeros(sshape)+0j
    ss = sd.reshape(-1,)
    ss[filtr] = fill
    sd = ss.reshape(sshape)
    y = np.fft.ifftn(sd)
    return y

x = np.arange(-23, 45, 1)
y = x**2+3*x*np.sin(x)+(x**2)*np.sin(x/10)#*x_
X = x
Y = np.array(list(y)*2)
filtr,fill,sshape = packageItFFT(Y,40)
y_pred = depackageFFT(sshape,filtr,fill)[:len(x)].real
plt.plot(x,y,color = "blue",label = "actual")
plt.scatter(x,y_pred,color ="green",label="recovered after iFFT")
plt.title("FFT approximation")
plt.legend()
print("number of parameters to compress the data using FFT",len(fill))
```

```
number of parameters to compress the data using FFT 15
```



We can then use this training data to interpolate to the test set.
But still it does not solve the problem of larger training data requiring finding different points that are near to test points that is still computationally expensive.

One way is to decode the FFT to only those parts of the domain that are relevant for prediction, but still it would require decoding differently for different prediction points and also there are no libraries that support this out of the box for easier implementation.

We therefore look out for a novel approach → we can use fourier Series but it becomes computationally hard for higher dimensions. We therefore find a map that projects the higher dimensions to lower dimension (1d) while preserving locality closeness property.

```
import numpy as np
import functools
import operator
# !pip install sympy
from sympy import fourier_series
import numpy as np
import pandas as pd
from sympy import fourier_series, pi

import matplotlib.pyplot as plt

import sklearn.datasets
from sklearn.datasets import make_blobs
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KDTree

from keras.datasets import cifar10
from keras.utils import to_categorical
import tensorflow as tf
from tensorflow.keras.datasets import mnist
def mapd(x,y=1):
    def m1(x):
#         print(x)
        return np.polyval(x,0.01j)
    def m2(x):
        return np.polyval(x,-0.01j)
    return y*(np.array(list(map(m1,x))) - 1j*np.array(list(map(m1,x))))+x.sum(axis=1)
    sd = ((np.array(list(map(m1,x))) + np.array(list(map(m2,x)))))*((np.array(list(map(m1,x))) - np.array(list(map(m
    pp = np.linalg.norm(x,axis=1)## the power seems to inert the category values graph around 1
    print(x.shape,sd.shape)
    return (-1*(sd+pp*1j)**2).real#np.array(list(map(m2,sd+pp*1j)))#np.polyval(x,sd+pp*1j)#(np.log(((((pp))+(((sd)**
    return ((abs(pp))*abs(sd)/abs(sd)**(1/((pp)*abs(sd)-pp)))## inverting about 1 as singularity and logging it as g
    return abs(sd)**(1/((pp)*abs(sd)-pp))
```

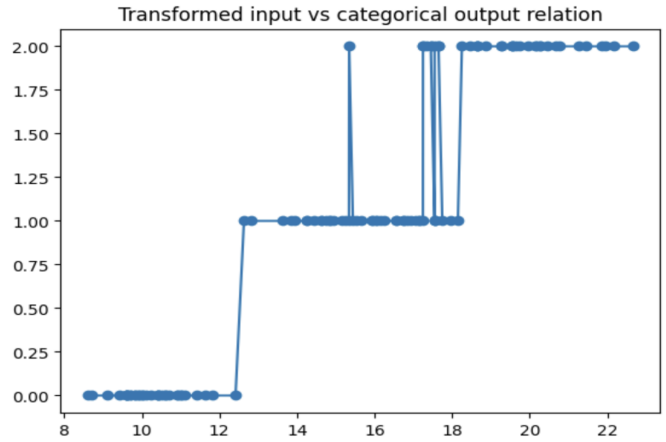
To test wheather the function is a good proxy for the required map, we test if closer points are mapped to simmilar values and output space or the map is fairly one to one.

```
We take x and some random deviations around it and check their values with getting closser to x in next round
x [1 4 0 4 2 4]
[19.01960004-3.97960004j 19.01960004-3.97960004j 19.01960004-3.97960004j
 19.01960004-3.97960004j 19.01960004-3.97960004j 19.01960004-3.97960004j]

_____
[22.76622607-4.40504346j 23.27667857-4.0684495j  23.6833066 -4.67860704j
 22.61153218-4.76593823j 22.31543072-4.16553097j]

_____
[19.34640835-4.02135177j 19.33278581-4.01500311j 19.30544679-4.03888792j
 19.35185767-3.98197855j 19.358463  -3.98988318j]

_____
[19.05578601-3.98584601j 19.06514492-3.98810171j 19.0594816 -3.98685329j
 19.06544503-3.98397141j 19.03636848-3.98197219j]
Clearly as the values get closer the output space gets closer


x [-1  4  0  4  2  4]
[17.01960004-3.97960004j 17.01960004-3.97960004j 17.01960004-3.97960004j
 17.01960004-3.97960004j 17.01960004-3.97960004j 17.01960004-3.97960004j]

_____
[21.73893871-4.52245904j 19.32471479-3.9957295j  19.16234537-4.00515135j
 19.58128945-4.1414297j  20.95183099-4.39313144j]

_____
[17.26935026-3.9857595j  17.24426387-3.98392969j 17.39770407-4.03685863j
 17.34432203-3.99758441j 17.34610252-4.0400769j ]

_____
[17.08180882-3.98765062j 17.05303797-3.98530278j 17.04491386-3.98136951j
 17.04661284-3.98325978j 17.04522874-3.98150367j]


x [1 4 1 4 2 4]
[20.01959904-3.97960104j 20.01959904-3.97960104j 20.01959904-3.97960104j
 20.01959904-3.97960104j 20.01959904-3.97960104j 20.01959904-3.97960104j]

_____
[23.92838865-4.8828214j  23.89011838-4.20463687j 23.56215745-4.64823283j
 22.73445215-4.52353576j 22.42914191-4.02254206j]

_____
[20.21715356-4.02599011j 20.26410971-3.98552529j 20.35016745-4.01805947j
 20.42713632-4.04220089j 20.60669054-4.06623287j]

_____
[20.04175886-3.98136034j 20.06376424-3.9873433j  20.04829745-3.98009998j
 20.04189935-3.98689209j 20.04474407-3.98045539j]


x [ 1 -4  0  4  2  4]
[11.01959996-3.97959996j 11.01959996-3.97959996j 11.01959996-3.97959996j
 11.01959996-3.97959996j 11.01959996-3.97959996j 11.01959996-3.97959996j]

_____
[13.9988703 -4.67108777j 15.79656732-4.69479101j 15.40167679-4.47349032j
 14.99289652-4.1043858j  14.41262406-4.45650863j]

_____
[11.38788697-4.05393791j 11.3509412 -3.98810743j 11.37231569-4.04322161j
 11.41911887-4.05932073j 11.31804791-4.00310677j]

_____
[11.04778581-3.98176815j 11.06264826-3.98188386j 11.05115437-3.98412594j
 11.04653249-3.98730114j 11.05536809-3.9838441j ]


x [ 1  4  0 -7  2  4]
[8.02070004-3.98070004j 8.02070004-3.98070004j 8.02070004-3.98070004j
 8.02070004-3.98070004j 8.02070004-3.98070004j 8.02070004-3.98070004j]

_____
[10.43570848-4.05988354j  9.6459984 -3.98716042j 11.51876373-4.7979329j
 12.22192199-4.89894314j 11.29691438-4.29581776j]

_____
[8.28132331-4.0605732j  8.27368416-3.98339111j 8.33860075-4.0269223j
 8.26130589-3.98385236j 8.21626148-3.98131404j]

_____
[8.0632267 -3.98766022j 8.04781323-3.98343136j 8.04808728-3.98318241j
 8.06457693-3.98875717j 8.04615409-3.98196871j]


Different values are mapped uniquely
```

Once we have the map we can plot the relation between this new input and output::

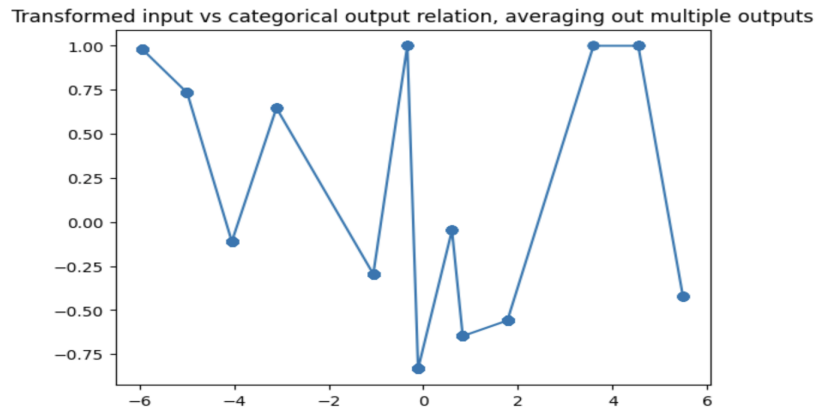
```
## Iris dataset
X, Y = sklearn.datasets.load_iris(return_X_y=True)
x_train,x_test,Y_train,Y_test = train_test_split(X,Y)
## We normalize the input for better mapping
xw = (x_train - x_train.mean(axis=0))/x_train.std(axis=0)
xx = mapd(x_train)
xc = sorted(xx)
inid = list(range(len(xc)))
inid.sort(key = lambda x:xx[x])
yc = Y_train[inid]
plt.plot(xc,yc,label='')
plt.scatter(xc,yc)
plt.title("Transformed input vs categorical output relation")
# plt.plot(xct,yct)
# plt.scatter(xct,yct)
```

```
Text(0.5, 1.0, 'Transformed input vs categorical output relation')
```




```
## Iris dataset
X, Y = sklearn.datasets.fetch_openml("titanic", return_X_y=True, as_frame=False)
x_train,x_test,Y_train,Y_test = train_test_split(X,Y)
Y_train = np.array(list(map(int,Y_train)))
Y_test = np.array(list(map(int,Y_test)))
## We normalize the input for better mapping
xw = (x_train - x_train.mean(axis=0))/x_train.std(axis=0)
xx = mapd(x_train)
xc = sorted(xx)
inid = list(range(len(xc)))
inid.sort(key = lambda x:xx[x])
yc = Y_train[inid]
uniqueDict = {}
for i in range(len(xc)):
    uniqueDict[xc[i]] = []
for i in range(len(xc)):
    uniqueDict[xc[i]].append(yc[i])
for i in uniqueDict:
    uniqueDict[i]=np.array(uniqueDict[i]).mean()
yc_ = []
for i in xc:
    yc_.append(uniqueDict[i])
yc = yc_
plt.plot(xc,yc,label='')
plt.scatter(xc,yc)
plt.title("Transformed input vs categorical output relation, averaging out multiple outputs")
```

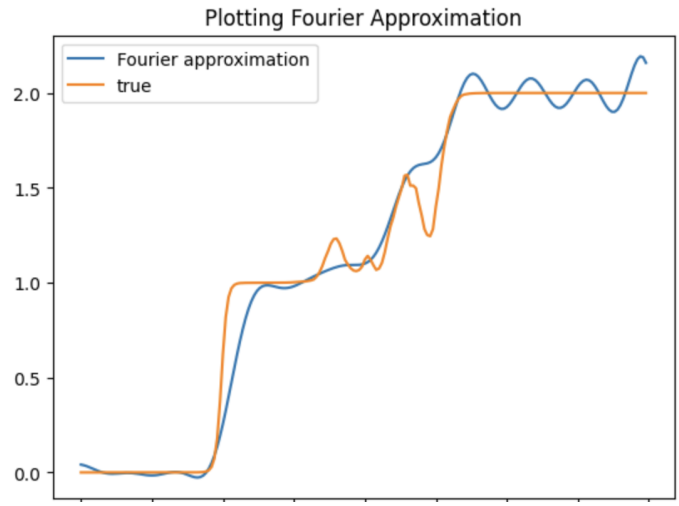
```
Text(0.5, 1.0, 'Transformed input vs categorical output relation, averaging out multiple outputs')
```



We then use the fourier series to approximate this 1d function and get the coefficients.

```
#function that computes the real fourier couples of coefficients (a0, 0), (a1, b1)...(aN, bN)
def compute_real_fourier_coeffs(func, N,P):
    result = []
    for n in range(N+1):
        an = (2./P) * spi.quad(lambda t: func(t)[0] * np.cos(2 * np.pi * n * t / P), 0, P)[0]
        bn = (2./P) * spi.quad(lambda t: func(t)[0] * np.sin(2 * np.pi * n * t / P), 0, P)[0]
        result.append((an, bn))
    return np.array(result)

#function that computes the real form Fourier series using an and bn coefficients
def fit_func_by_fourier_series_with_real_coeffs(t, AB):
    result = 0.
    A = AB[:,0]
    B = AB[:,1]
    for n in range(0, len(AB)):
        if n > 0:
            result +=  A[n] * np.cos(2. * np.pi * n * t / P) + B[n] * np.sin(2. * np.pi * n * t / P)
        else:
            result +=  A[0]/2.
    return result

AB = compute_real_fourier_coeffs(f, 15,25)
#AB contains the list of couples of (an, bn) coefficients for n in 1..N interval.

y_approx = fit_func_by_fourier_series_with_real_coeffs(t_range, AB)
plt.plot(y_approx[:200],label="Fourier approximation")
plt.plot(y_true[:200],label = "true")
plt.title('Plotting Fourier Approximation')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x2918c77c0>
```



We can then use these coefficients to make predictions on new data (test data)..
The accuracy increases as we increase the degree of approximation but also increase the computation time..

A implementation note -

The training data was extended to make it periodic so then we have a better fourier approximation, this was possible as we have finite domains cause we are only interested in interpolation not extrapolation..

Finalizing to a complete trainable model::

1) Normalize and convert input data to lower 1 dimensional input
2) Use numerical or KNN_classify to interpolate in between the curve
3) Use Fourier series to approximate the curve to a few parameters
4) Use that FS to predict values

```
##3 To soothen out the whole process into just one function::

def reduceSupervised(x_train,Y_train,N=8,isR = False):
    xx = conv(x_train)
    xc,yc = sortItOut(xx,Y_train)
    xc,yc = uM(xc,yc)
    f = giveInterpolFunc(xc,yc,isR)
    return compute_real_fourier_coeffs(f,N,round(xc[-1].real))
```

```
## Iris dataset
X, Y = sklearn.datasets.load_iris(return_X_y=True)
x_train,x_test,Y_train,Y_test = train_test_split(X,Y)
AB = reduceSupervised(x_train,Y_train,25)
```

```
## predicting
y_pred = np.array(list(map(round,((predd(x_test,AB))))))
```

```
## accuracy
acc = 1 - (sum(abs(y_pred-Y_test)))/len(y_pred)
acc
```

```
0.8421052631578947
```

```
X, Y = sklearn.datasets.fetch_openml("titanic", return_X_y=True, as_frame=False)
x_train,x_test,Y_train,Y_test = train_test_split(X,Y)
Y_test = (np.array(list(map(int,Y_test)))+1)/2
Y_train = (np.array(list(map(int,Y_train)))+1)/2
AB = reduceSupervised(x_train,Y_train)
```

```
## predicting
y_pred = np.array(list(map(round,((predd(x_test,AB))))))
```

```
## accuracy
acc = 1 - (sum(abs(y_pred-Y_test)))/len(y_pred)
acc
```

```
0.7477313974591652
```

Conclusion:

Spatial closeness captures important and enough information regarding categorization of different labels, Using projection maps and Fourier Series we were able to capture this closeness property and its relation with required labels giving us a method or model that does not require a large training time yet is decent on performance.